Discovery Executive Summary

Project: discovery-14-aug · **Generated:** 14/08/2026, 12:29:26

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 8 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Code Quality & Complexity Analysis	—
3	Frontend Modernization Analysis	—
4	Backend Modernization Analysis	—
5	Testing & Quality Assurance Analysis	—
6	Security Analysis	—
7	Performance & Sustainability Analysis	—
8	Technical Debt	—

---

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo with three layers: a Laravel backend (997 LOC across 6 controllers, 4 models, 2 services), a React 19 / TypeScript SPA frontend (983 LOC across 4 pages, 4 components, 1 hook, 1 store), and a Node.js/Express dev-api (832 LOC). The architecture exhibits several moderate-to-high-risk hotspots. The most severe issue is the complete absence of a Repository layer — all 25+ Eloquent ORM access points are scattered directly through controllers and services with no data-access abstraction. Additional concerns include business logic embedded in controllers (reachability-percentage calculations duplicated across `ConnectController`, `LegacyReportController`, and `RealTimeTestService`), cross-domain model access in `LegacyReportController` and `DashboardController`, and on the frontend, oversized page components with mixed concerns plus a legacy class component with an intentional resource leak. The dominant risk is change amplification: modifying the reachability formula requires synchronized edits across 3 backend files and 1 dev-api file. **Layers covered:** Backend (6 PHP controllers, 4 models, 2 services, 1 legacy mapper — 18 PHP source files), Frontend (4 pages, 4 components, 1 hook, 1 store, 1 API client, 1 types file — 12 TS/TSX/JSX source files), Dev-API (5 JS source files).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 avg (6 controllers); but `LegacyReportController` contains duplicated business logic, `extract()` , and KPI math	Moderate
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	14 direct model access points across controllers	Moderate
H3	Missing Repository Pattern	Direct DB/ORM access points outside repositories	<10	10–20	>20	25+ (0 repository classes; all ORM access is direct)	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (`LegacyDataMapper` with unsafe `extract()`)	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% (no raw SQL; all queries use Eloquent)	Good

H7	God Classes	Classes >1000 LOC	0	1–3	>3	0	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (LegacyReportController imports Connect + Discovery models; DashboardController imports both domains)	Moderate
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	0% (4 tables with clear domain ownership)	Good
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	113 avg across 8 components/pages; but ConnectPage=222 LOC, DiscoveryPage=176 LOC	Moderate
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	2	Good
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0	Good
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	1 level	Good
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	1 (`LegacyMonitorPoller.jsx` — class component with interval leak)	Moderate

1.4 Actions Required

Hotspot	Action	Rating	Priority
H3 — Missing Repository Pattern	Create `ConnectMonitorRepository`, `ConnectCheckResultRepository`, `DiscoveryJobRepository`, `DiscoveryNodeRepository` with interfaces; bind in `AppServiceProvider`; migrate all 25+ direct Eloquent calls to use repositories	High Risk	Critical
H1 — Fat Controllers	Extract reachability calculation into `ReachabilityCalculator` service; remove duplicate `buildTree()` from `LegacyReportController`; replace `extract()` with explicit array access	Moderate	High
H2 — Missing Service Layer	Create `ConnectService`, `DiscoveryService`, `DashboardService`; move CRUD and KPI logic from controllers into services; replace hardcoded KPIs in `DashboardController`	Moderate	High
F5 — Legacy / Inconsistent Patterns	Convert `LegacyMonitorPoller` class component to function component with `useEffect` cleanup; add Error Boundary around `LegacyDashboardWidget`; rename `.jsx` to `.tsx`	Moderate	High
H5 — Shared Utility Abuse	Replace `extract()` calls in `LegacyDataMapper` with explicit destructuring; register class in container for DI instead of `new` instantiation	Moderate	Medium
H8 — Domain Boundary Violations	Move models into domain namespaces (`App\\Modules\\Connect\\Models`, `App\\Modules\\Discovery\\Models`); create domain service interfaces for cross-domain queries	Moderate	Medium
F1 — Business Logic in Components	Extract React Query hooks into per-domain custom hooks; centralize query invalidation in `useRealtimeTest`; split `ConnectPage` and `DiscoveryPage` into sub-components	Moderate	Medium

1.5 Expected Outcomes

- **Separation of concerns:** Controllers become thin HTTP translators; all business logic lives in testable service and domain classes, eliminating the 4-location reachability formula duplication.
- **Testability:** Repository interfaces enable unit testing with in-memory fakes; services can be tested without HTTP or database overhead.
- **Independent module evolution:** Bounded contexts with published interfaces allow Connect and Discovery teams to evolve their schemas and logic independently, paving the way for microservice extraction.
- **Frontend maintainability:** Custom hooks and sub-components keep page files under 100 LOC; centralized invalidation removes duplicated cache management across pages.
- **Reduced regression risk:** Eliminating `extract()`, fixing the interval leak, and adding Error Boundaries remove three categories of runtime bugs (variable injection, memory leaks, unhandled crashes).","stop_reason":"end_turn","session_id":"f70f5ff0-9149-404e-97d1-baaae082c58a","total_cost_usd":2.351398,"usage":{"input_tokens":19,"cache_creation_input_tokens":108982,"cache_read_input_tokens":1107098,"output_tokens":27894,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":108982,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":2293,"cache_read_input_tokens":98567,"cache_creation_input_tokens":10415,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":10415},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":10509,"outputTokens":15,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.010584,"contextWindow":200000,"maxOutputTokens":3200
opus-4-6":{"inputTokens":19,"outputTokens":27894,"cacheReadInputTokens":1107098,"cacheCreationInputTokens":108982,"webSearchRequests":0,"costUSD":2.340814,"contextWindow":200000,"maxOutput
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"8c5e9156-6bc8-440d-913f-63c62f0c0c88"}

---

2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

The Klearcom platform codebase is a small monorepo (~3,008 LOC) with a Laravel backend, a React/TypeScript frontend, and a Node.js development API. Overall code quality is reasonable for an early-stage project, but one structural issue rises to High Risk: the `ConnectPage` React component is a 214-line single function that mixes form state, three TanStack Query subscriptions, mutation logic, event handlers, and deeply nested conditional JSX — exceeding the 200 LOC function threshold. Cyclomatic complexity is Moderate, peaking at ~12 branches in the same component. Duplicate code sits at ~5% overall, driven by identical `buildTree` implementations in three locations and a reachability-calculation block copied across `ConnectController`, `RealTimeTestService`, and the dev-API. Git churn and ownership metrics are healthy (3 total commits, single author), so stability risks are minimal. One additional hotspot — a resource-leak anti-pattern in `LegacyMonitorPoller.jsx` — was identified. Analysis covered both backend (26 PHP files) and frontend (15 TS/TSX/JSX files) layers in full.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	~12 (ConnectPage.tsx)	Moderate
H2	Large Classes	Largest class/file LOC	<300	300–1000	>1000	276 LOC (dev-api/src/server.js)	Good
H3	Large Functions	Largest function/method LOC	<50	50–200	>200	214 LOC (ConnectPage)	High Risk
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~1.3% (buildTree ×3, reachability calc ×3)	Good
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~5% (structural + copy-paste)	Moderate
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	2 (server.js, api.php, ConnectController.php)	Good
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	1 (App.tsx)	Good
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	100% (single author: ksabai-gl)	Good
H9	Resource Leak (additional)	Components with missing cleanup	0	1	>1	1 (LegacyMonitorPoller.jsx)	Moderate

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	40	10.0
Code Churn	25%	10	2.5
Defect Density	20%	15	3.0
Class/Function Size	15%	70	10.5
Business Logic Duplication	10%	40	4.0
Developer Ownership Risk	5%	5	0.25
Hotspot Score	**100%**		**30 / 100**

2.5 Actions Required

Hotspot	Action	Rating	Priority
H3 — Large Functions	Decompose ConnectPage (214 LOC) and DiscoveryPage (166 LOC) into sub-components and custom hooks, each under 80 LOC	High Risk	Critical
H1 — High Cyclomatic Complexity	Extract conditional rendering sections into guard components; flatten nested ternaries with early returns	Moderate	Medium
H5 — Duplicate Code	Create shared page-layout composition and extract common query/mutation hooks; remove or migrate LegacyDashboardWidget	Moderate	Medium
H9 — Resource Leak	Add componentWillUnmount cleanup to LegacyMonitorPoller or convert to function component; add Error Boundary for LegacyDashboardWidget	Moderate	Medium
H4 — Business Logic Duplication	Consolidate reachability calculation into ConnectMonitor model method; extract buildTree into shared service or trait	Good	Low

2.6 Expected Outcomes

- **Lower defect risk on page changes:** Decomposing ConnectPage and DiscoveryPage into focused sub-components reduces the number of test paths per unit from ~12 to ~3–4, cutting regression risk on UI changes.
- **Single-source business rules:** Consolidating reachability calculation and tree-building logic eliminates the risk of formula drift across three codebases (Laravel, dev-API, controller).
- **Safer reviews:** Smaller, focused components and hooks are easier to review in PRs — reviewers can assess one concern at a time rather than parsing a 214-line monolith.
- **Eliminated resource leaks:** Fixing the interval leak in LegacyMonitorPoller prevents accumulated network traffic and React warnings in development, improving runtime stability.
- **Clearer architecture for growth:** Shared layout patterns and extracted hooks establish a composition model that scales cleanly as Discovery and Connect modules add features (filtering, export, bulk operations).","stop_reason":"end_turn","session_id":"6402def5-516a-462a-b896-fc4f1bb199e7","total_cost_usd":2.613769,"usage":{"input_tokens":23,"cache_creation_input_tokens":115033,"cache_read_input_tokens":1381344,"output_tokens":30544,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":115033,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":2070,"cache_read_input_tokens":106636,"cache_creation_input_tokens":8397,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":8397},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":8967,"outputTokens":17,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.009052,"contextWindow":200000,"maxOutputTokens":32000}opus-4-6":{"inputTokens":23,"outputTokens":30544,"cacheReadInputTokens":1381344,"cacheCreationInputTokens":115033,"webSearchRequests":0,"costUSD":2.604717,"contextWindow":200000,"maxOutput[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"fab96261-ae48-461e-938d-6b0cfef8047c"}

---

3. Frontend Modernization Analysis

EXECUTIVE SUMMARY

The FSDKC frontend is a small React 19 + TypeScript application (983 LOC across 15 files) built with modern tooling — Vite, Zustand, and TanStack React Query — and TypeScript strict mode enabled. Despite the modern stack, it suffers from significant structural issues: heavy page-level duplication between DiscoveryPage and ConnectPage, no authentication or route guards, no ESLint enforcement, 32 inline-style instances with hardcoded magic values, missing browser compatibility configuration, and 3 high-severity CVEs in react-router-dom. One legacy class component (LegacyMonitorPoller) leaks an interval due to missing lifecycle cleanup, and the app has no React Error Boundaries, meaning an uncaught throw in LegacyDashboardWidget crashes the entire UI. The centralized API client and React Query adoption are bright spots, but architectural boundaries are absent and the module folders are empty placeholders.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~22% (2 of 9 structurally duplicated)	High Risk
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	89% (8 of 9 functional)	Moderate
H3	Massive Components	Largest component LOC	<200	200–500	>500	222 LOC (ConnectPage.tsx)	Moderate
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	22% (2 of 9)	Good
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	1 level	Good
H6	Weak Frontend Architecture	Feature modules with clean boundaries %	>80%	50–80%	<50%	0% (module folders are empty)	High Risk
H7	Missing Component Inventory	Shared component % of total	>30%	15–30%	<15%	44% (4 of 9 in components/)	Good
H8	No Design System	Inline-style / magic-value occurrences	0–5	6–20	>20	32 inline styles	High Risk
H9	Routing Structure Weakness	Protected routes with guards %	100%	80–99%	<80%	0% (no auth guards)	High Risk
H10	No API Integration Layer	API calls in service layer %	>90%	70–90%	<70%	100% (all via api/client.ts)	Good
H11	Poor Data Caching	Data-fetching points with caching %	>70%	40–70%	<40%	67% (10 of 15 via React Query)	Moderate
H12	Weak Frontend Auth	Token storage + routes guarded	httpOnly + 100%	One gap	Both gaps	No auth system; 0% guarded	High Risk
H13	Frontend Security Vulnerabilities	XSS-risk + hardcoded secrets count	0 each	1–3 total	>3 total	0 each	Good
H14	Frontend Performance Gaps	Initial JS bundle size (gzipped)	<250KB	250–500KB	>500KB	~75KB estimated (minimal deps)	Good

H15	Browser Compatibility Gaps	Browserslist + polyfills configured	Both present	One missing	Both missing	Both missing	High Risk
H16	Frontend Code Quality	ESLint in CI + TypeScript strict	Both Yes	One Yes	Both No	No ESLint + strict: true	Moderate
H17	Technical Debt & Dependencies	Critical/High CVEs found	0	1–3	>3	3 high CVEs (react-router-dom)	Moderate
H18	Missing Error Boundaries (additional)	Error boundary coverage (target: all feature routes wrapped)	All routes wrapped	Some routes wrapped	No error boundaries	0 error boundaries	High Risk

3.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — UI Component Duplication	Extract shared PageShell, TranscriptList, and DataTable components; remove LegacyDashboardWidget	High Risk	Critical
H6 — Weak Frontend Architecture	Move feature code into src/modules/ with barrel exports; add import boundary rules	High Risk	Critical
H9 — Routing Structure Weakness	Add RequireAuth guard, React.lazy code splitting, and 404 fallback route	High Risk	Critical
H12 — Weak Frontend Auth	Implement OAuth 2.0/OIDC auth with httpOnly cookies and role-based access control	High Risk	Critical
H18 — Missing Error Boundaries	Add ErrorBoundary per route and top-level fallback; fix uncaught throw in LegacyDashboardWidget	High Risk	Critical
H8 — No Design System	Define spacing/typography tokens; create utility CSS classes; eliminate 32 inline styles	High Risk	High
H15 — Browser Compatibility	Add .browserslistrc, Vite build targets, and Autoprefixer	High Risk	High
H16 — Frontend Code Quality	Add ESLint with react-hooks plugin and integrate into CI	Moderate	High
H17 — Technical Debt	Run npm audit fix for react-router CVEs; schedule quarterly dependency audits	Moderate	High
H2 — Legacy Class Components	Convert LegacyMonitorPoller to functional component with useQuery	Moderate	Medium
H3 — Massive Components	Split ConnectPage and DiscoveryPage into sub-components	Moderate	Low
H11 — Poor Data Caching	Convert LegacyDashboardWidget and LegacyMonitorPoller to React Query	Moderate	Medium

3.5 Expected Outcomes

- **Shared component library** (PageShell, TranscriptList, DataTable) reduces page-level duplication from ~22% to <5%, cutting maintenance cost for new module additions.
- **Feature module boundaries** with barrel exports and import rules enable independent feature development and testing, preventing cross-module coupling as the app scales.
- **Authentication + route guards** prevent unauthorized access to voice infrastructure controls (IVR discovery, TFN monitoring), closing the most critical security gap.
- **Error Boundaries** per route isolate rendering failures to individual pages, preventing a single widget error from white-screening the entire application.
- **ESLint with react-hooks plugin** catches stale closure bugs, missing effect dependencies, and hook rule violations at lint time rather than in production.
- **CVE remediation** via `npm audit fix` eliminates 3 high-severity react-router vulnerabilities (open redirect, XSS, DoS).
- **Design tokens + utility classes** replace 32 inline style instances with a single source of truth for spacing and typography, enabling consistent brand changes.
- **Browserslist + build targets** ensure the production bundle works across the target browser matrix, preventing silent breakage on older Safari/Firefox ESR.","stop_reason":"end_turn","session_id":"412259b9-9c0e-4551-b91d-f51a35f6ffd2","total_cost_usd":2.3310015,"usage":{"input_tokens":18,"cache_creation_input_tokens":94639,"cache_read_input_tokens":898133,"output_tokens":37007,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":94639,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":2567,"cache_read_input_tokens":82798,"cache_creation_input_tokens":11841,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":11841},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":10215,"outputTokens":13,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.010280000000000001,"contextWindow":200000,"maxOutpu
opus-4-6":{"inputTokens":18,"outputTokens":37007,"cacheReadInputTokens":898133,"cacheCreationInputTokens":94639,"webSearchRequests":0,"costUSD":2.3207215,"contextWindow":200000,"maxOutputT
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"bf468997-791b-4362-89a9-383ad19be5f3"}

---

4. Backend Modernization Analysis

EXECUTIVE SUMMARY

The Klearcom backend is a PHP 8.3 / Laravel 12 monolith serving a Voice & Telecom QA platform with two feature modules (Discovery and Connect) backed by MariaDB and MongoDB. The most severe gaps are the complete absence of authentication and authorization on all 19 API endpoints, the use of `extract()` on raw user input creating untraceable variable injection, and database schema managed via raw SQL init scripts with no migration framework and no FK indexes. Controllers contain inline ORM queries and duplicated business logic (reachability calculation appears in three places; `buildTree` is copy-pasted across two controllers) instead of delegating to a service layer. No rate limiting, no security headers, no caching layer, no linter enforcement in CI, and CORS is configured as wildcard `*` — the backend is functionally unprotected. The API surface has no OpenAPI spec, no versioning, and no contract tests.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	3 (extract calls; 1 on raw $request->all())	Moderate
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	0	Good
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	~15% (only MongoService queries in a service; all Eloquent ORM in controllers)	High Risk
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	0 (static calls are Eloquent facade usage, not custom singletons)	Good
H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	4 controllers with inline business logic + duplicated logic across files	Moderate
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	0% — no OpenAPI spec, no documentation	High Risk
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% — no spec, no versioning, no contract tests, no API linting	High Risk
H8	Weak Application Architecture	Modules following declared architecture %	>80%	50–80%	<50%	~33% — only MongoController and StreamController delegate to services (2 of 6)	High Risk
H9	Missing Module Inventory	Circular dependency count	0	1–3	>3	0	Good
H10	Database Schema Weakness	FK indexes % + migrations with rollback %	Both >90%	One <90%	Both <90%	0% explicit FK indexes on non-FK-constraint columns (parent_id) + 0% migration rollback (raw SQL init, no Laravel migrations)	High Risk
H11	Middleware Weakness	Required middleware present + ordered %	100%	80–99%	<80%	~20% — only CORS present; no auth, no rate limiting, no security headers, no request logging	High Risk
H12	Auth & Authorization Weakness	Protected routes guarded % + hashing algo	100% + bcrypt/argon2	One gap	Both bad	0% routes guarded + no password hashing (no auth system)	High Risk
H13	Backend Security Vulnerabilities	Injection + hardcoded secrets count	0 each	1–3 total	>3 total	7 total (1 extract-injection on $request->all(), wildcard CORS, 4 plaintext passwords in docker-compose + CI, hardcoded KPI values)	High Risk
H14	Performance & Caching Gaps	N+1 patterns found	0	1–5	>5	1 (LegacyReportController::carrierSummary loops monitors with per-monitor query) + zero caching	Moderate
H15	Outdated & Vulnerable Dependencies	Critical/High CVEs found	0	1–3	>3	0 (no composer.lock to audit; only 3 production deps, all latest)	Good
H16	Secrets & Configuration in Source	Hardcoded secrets / .env committed	0	1–2	>2	3 (.env.example with DB_PASSWORD=secret, docker-compose.yml with 3 plaintext passwords, CI workflow with hardcoded passwords)	High Risk
H17	Backend Code Quality	Linter in CI + max cyclomatic complexity	Both good	One gap	Both bad	No linter in CI (phpstan in require-dev but not run) + no complexity enforcement	High Risk
H18	Missing Transaction	Multi-step DB writes without	0	1–3	>3	4 (RealTimeTestService multi-model writes in runDiscoveryTest and runConnectTest)	<span class="rating rating-

	Boundaries (additional)	transaction wrapping					moderate">Moderate

4.4 Actions Required

Hotspot	Action	Rating	Priority
H12 — Auth & Authorization	Add Laravel Sanctum auth middleware to all non-health routes; add user password column; implement object-level authorization policies	High Risk	Critical
H11 — Middleware Weakness	Add auth, rate limiting, security headers, request logging middleware; restrict CORS origins	High Risk	Critical
H1 — Dynamic Variable Creation	Remove all extract() calls; use Form Request validation with explicit field access	Moderate	Critical
H13 — Security Vulnerabilities	Eliminate extract-injection, move secrets to env/vault, use GitHub Actions secrets in CI, remove hardcoded KPIs	High Risk	Critical
H10 — Database Schema Weakness	Convert raw SQL to Laravel migrations with rollback; add index on parent_id	High Risk	Critical
H3 — Direct ORM in Controllers	Create Repository classes for all 4 models; move all Eloquent queries out of controllers	High Risk	High
H8 — Weak Architecture	Enforce service-layer pattern across all controllers; add deptrac/PHPStan rules to prevent Model calls from Controllers	High Risk	High
H5 — Missing Service Layer	Extract ReachabilityService, IvrTreeService, DashboardService to eliminate duplicated logic	Moderate	High
H16 — Secrets in Source	Move docker-compose and CI passwords to Docker secrets and GitHub Actions secrets	High Risk	High
H7 — Missing API Governance	Generate OpenAPI spec, add API versioning, add contract tests and Spectral linting to CI	High Risk	High
H6 — API Sprawl	Standardize response envelope, document all endpoints, add version prefix	High Risk	Medium
H17 — Code Quality	Configure phpstan.neon at level 6+, add phpstan + Pint to CI, replace non-deterministic tests	High Risk	High
H14 — Performance & Caching	Add eager loading for N+1, introduce Redis caching for dashboard and carrier summary	Moderate	Medium
H18 — Missing Transactions	Wrap multi-model writes in DB::transaction; add error handling in dispatch closures	Moderate	Medium

4.5 Expected Outcomes

- **Authentication eliminates unauthorized access:** Adding Sanctum auth middleware to all routes prevents anonymous users from triggering telephony tests (which incur carrier costs) and accessing sensitive monitoring data.
- **extract() removal eliminates variable injection risk:** Replacing `extract($request->all())` with typed Form Requests closes the most critical injection vector, making data flow explicit and auditable.
- **Service layer enables logic reuse:** Extracting `ReachabilityService` and `IvrTreeService` eliminates the three duplicated reachability calculations and two duplicated `buildTree` methods, creating single sources of truth.
- **Repository layer decouples persistence:** Moving ORM queries into repositories allows controllers to be tested without a database and enables future storage changes without touching business logic.
- **Migration framework prevents schema drift:** Converting raw SQL to Laravel migrations with `down()` methods enables version-controlled, rollback-capable schema changes and eliminates environment drift.
- **API governance prevents breaking changes:** An OpenAPI spec with contract tests in CI ensures that API changes are detected before they reach consumers, and versioning enables non-breaking API evolution.
- **Redis caching reduces database load:** Caching dashboard KPIs and carrier summaries eliminates repetitive aggregate queries, reducing database load by an estimated 80%+ on read-heavy endpoints.
- **CI quality gates catch regressions early:** Enforcing PHPStan level 6+ and Laravel Pint in CI catches type errors, undefined variables, and style violations before they reach production.","stop_reason":"end_turn","session_id":"69585a1a-58a3-4cee-a538-26f8b0dab47b","total_cost_usd":2.42277,"usage":{"input_tokens":17,"cache_creation_input_tokens":109039,"cache_read_input_tokens":866972,"output_tokens":35536,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":109039,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":3042,"cache_read_input_tokens":96250,"cache_creation_input_tokens":12789,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":12789},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":10339,"outputTokens":14,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.010409,"contextWindow":200000,"maxOutputTokens":3200(
opus-4-6":{"inputTokens":17,"outputTokens":35536,"cacheReadInputTokens":866972,"cacheCreationInputTokens":109039,"webSearchRequests":0,"costUSD":2.4123609999999998,"contextWindow":200000,"
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"391a1bac-4f76-4172-95f6-8b865c36f88b"}

5. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

The Klearcom platform has a critically thin test suite. The backend (Laravel 12 / PHP 8.3) ships with PHPUnit 11 but contains only 2 test files (3 test methods) — none of which exercise real business logic; they assert hardcoded arrays and random integers. All 6 API controllers, both service classes (RealTimeTestService, MongoService), the LegacyDataMapper, and all 4 Eloquent models are completely untested. The frontend (React 19 / TypeScript / Vite) has zero test infrastructure — no Jest, Vitest, Testing Library, Cypress, or Playwright is installed, and zero test files exist for the 13 source files including pages, components, hooks, and the API client. CI runs PHPUnit on every PR but with `coverage: none`, and the frontend CI job only runs `npm run build` — no tests are executed. Estimated overall test coverage is **< 5%** (backend) and **0%** (frontend).

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	8 modules (6 controllers, 2 services)	High Risk
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	~3% (estimated, backend ~5%, frontend 0%)	High Risk
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (zero integration tests)	High Risk
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (0 of 16 endpoints)	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0	Good
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	Backend: runs, not required; Frontend: no test run	Moderate
H7	No Frontend Test Infrastructure (additional)	Frontend test framework installed + test files present	Framework + tests	Framework, no tests	No framework	No framework installed, 0 test files	High Risk
H8	Assertion-Free / Trivial Tests (additional)	Tests with no meaningful business assertions (target 0)	0	1–2	>2	3 (all existing test methods)	High Risk

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Write unit tests for RealTimeTestService, MongoService, LegacyDataMapper; write feature tests for all 6 API controllers	High Risk	Critical
H2 — Low Test Coverage	Enable coverage reporting in PHPUnit/CI; install Vitest for frontend; target 75% backend and 60% frontend coverage	High Risk	Critical
H3 — Missing Integration Tests	Add Laravel feature tests with RefreshDatabase; add MongoDB service to CI; test SSE streaming flow end-to-end	High Risk	Critical
H4 — Missing Contract Tests	Add assertJsonStructure tests for all 16 API endpoints; test validation rejection for mutation endpoints	High Risk	Critical
H6 — No CI Test Gate	Add frontend test step to CI; make both backend and frontend CI jobs required status checks on main	Moderate	Medium
H7 — No Frontend Test Infrastructure	Install Vitest + React Testing Library + JSDOM; write tests for useRealtimeTest hook, API client, pages, and store	High Risk	Critical
H8 — Assertion-Free / Trivial Tests	Replace all 3 trivial test methods with tests that exercise real application classes; remove tautological assertions	High Risk	High

5.5 Expected Outcomes

- **Critical business logic protected:** RealTimeTestService (IVR discovery, TFN reachability) and MongoService verified by unit tests, preventing silent regressions in the platform's core test execution and data pipeline.
- **API contracts enforced:** All 16 endpoints validated by contract tests, catching breaking response shape changes before they reach the frontend — especially the duplicated reachability calculation that could silently diverge.
- **Frontend safety net established:** Vitest + React Testing Library installed and wired into CI, covering the useRealtimeTest SSE hook, page components, and Zustand store — preventing UI regressions during refactors.

- **CI as a true quality gate:** Both backend and frontend test suites run on every PR with coverage reporting, and branch protection requires both jobs to pass before merge — no untested code reaches main.
- **Legacy code safely modernizable:** With tests around LegacyDataMapper's `extract()` patterns and LegacyReportController's unfiltered input, the upcoming tech-debt cleanup can proceed with confidence that behavior is preserved.","stop_reason":"end_turn","session_id":"f1ed663c-081c-4d1c-b3d5-3e962adfac72","total_cost_usd":1.2154175,"usage":{"input_tokens":63,"cache_creation_input_tokens":47906,"cache_read_input_tokens":769783,"output_tokens":13748,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":47906,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":1724,"cache_read_input_tokens":62014,"cache_creation_input_tokens":7716,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":7716},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":7366,"outputTokens":17,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.007451,"contextWindow":200000,"maxOutputTokens":32000}opus-4-6":{"inputTokens":63,"outputTokens":13748,"cacheReadInputTokens":769783,"cacheCreationInputTokens":47906,"webSearchRequests":0,"costUSD":1.2079665,"contextWindow":200000,"maxOutputT[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"573de886-9c94-4b71-b34a-2ad27e1d1925"}

---

6. Security Analysis

EXECUTIVE SUMMARY

The Klearcom platform has significant security weaknesses across both its PHP backend and Node.js dev-API. The most critical finding is the complete absence of authentication and authorization on all API routes — every endpoint (including data-mutating POST routes) is publicly accessible without any credential check. A wildcard CORS configuration (`allowed_origins: ['*']`) compounds this by allowing any origin to call the API. The backend uses PHP's unsafe `extract()` on unfiltered user input in `LegacyReportController`, enabling variable injection. The dev-API `bulk-import` endpoint accepts arbitrary payloads with zero validation or rate limiting. Hardcoded database credentials (`secret` / `root`) are committed in `docker-compose.yml`, `.env.example`, and the CI workflow. `APP_DEBUG=true` is set in committed configuration, which would leak stack traces and environment variables in production. No security headers (CSP, HSTS, X-Frame-Options) are configured on either the Nginx reverse proxy or the application. The frontend has no XSS sinks, no hardcoded secrets, and no browser-storage token issues, but lacks a Content Security Policy and uses a hardcoded `http://` fallback API URL. The CI pipeline has no SAST, dependency scanning, or branch protection evidence. Layers covered: backend (PHP/Laravel), dev-API (Node.js/Express), frontend (React/TypeScript), infrastructure (Docker/Nginx/CI).

6.1 Security Benchmark Ratings

#	Security KPI	Target	Good	Moderate	High Risk	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	3	High Risk
H2	High Vulnerabilities	0	<5	5–10	>10	5	Moderate
H3	Medium Vulnerabilities	low	<20	20–50	>50	3	Good
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	3.6/KLOC	High Risk
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	30% (7/10 categories with findings)	High Risk
H6	Critical/High Vulnerable Deps	0	0	1	>1	0	Good
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	<10%	Good
H8	End-of-Life Dependencies	0	0	1–5	>5	0	Good

6.5 Actions Required

Finding	Action	Rating	Priority
No Authentication/Authorization	Implement auth middleware (Sanctum/JWT) on all API routes; add login/registration; apply RBAC	High Risk	Critical
Wildcard CORS	Restrict `allowed_origins` to explicit frontend domain allow-list in both Laravel and Express	High Risk	Critical
Unsafe `extract()` on User Input	Replace all `extract()` calls with explicit array access in `LegacyReportController` and `LegacyDataMapper`	High Risk	Critical
Hardcoded Credentials	Use Docker secrets / env-var references; replace `.env.example` passwords with placeholders; use GitHub Actions secrets in CI	High Risk	High

APP_DEBUG Enabled	Set `APP_DEBUG=false` in `.env.example` and remove from `docker-compose.yml`	High Risk	High
No Security Headers	Add CSP, X-Frame-Options, X-Content-Type-Options, HSTS to Nginx config and frontend CSP meta tag	High Risk	High
No CSRF Protection	Implement token-based auth (inherent CSRF protection) or add VerifyCsrfToken middleware	High Risk	High
No Rate Limiting	Register RateLimiter in AppServiceProvider; apply throttle middleware; add express-rate-limit to dev-API	High Risk	High
Missing Frontend CSP	Add `<meta http-equiv=\"Content-Security-Policy\">` to index.html; change fallback URL to HTTPS	Moderate	Medium
No SAST/Dependency Scanning in CI	Add `composer audit`, `npm audit`, PHPStan, and Dependabot to CI pipeline; enable branch protection	Moderate	Medium
No Security Audit Logging	Add structured logging for auth events, data mutations, and API access patterns	Moderate	Medium

---

7. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

The Klearcom platform has significant runtime-performance deficiencies driven by quadratic-complexity tree-building algorithms (six sites across PHP and JavaScript), unbounded in-memory data growth (five high-memory sites including uncapped SSE event fetches and an ever-growing in-memory store), and infrastructure without resource constraints or autoscaling. The SSE streaming architecture uses inefficient polling that re-fetches all session events every 500ms instead of using change streams or cursor-based pagination, compounding both memory pressure and network overhead. Docker containers run with no CPU/memory limits, and the CI pipeline lacks any dependency or layer caching — every push compiles the MongoDB PHP extension from source and downloads all packages. The sustainability posture is partial: always-on containers with no right-sizing, energy-inefficient polling patterns, and no carbon-aware scheduling. The most urgent risks are algorithm efficiency (P1), memory utilization (P4), resource provisioning (P8), and build efficiency (P10), all rated High Risk.

7.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	6	High Risk
P2	Database Performance	Deferred → Backend Modernization (H14/H10)	—	—	—	See Backend Modernization	— (deferred)
P3	API Performance	Response-latency hotspots	0	1–5	>5	3	Moderate
P4	Memory Efficiency	High-memory sites	0	1–3	>3	5	High Risk
P5	CPU Efficiency	CPU-intensive operations	0	1–5	>5	2	Moderate
P6	Concurrency	Parallelizable work + pool sizing (blocking-I/O → Backend Modernization H14)	0	1–5	>5	3	Moderate
P7	Caching	Deferred → Backend Modernization H14 / Frontend Modernization H11	—	—	—	See those reports	— (deferred)
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	4	High Risk
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	5	Moderate
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	no caching	High Risk
P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	0	Good
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	partial	Moderate

7.5 Actions Required

Hotspot	Action	Rating	Priority
P1 Algorithm Efficiency	Replace recursive `buildTree()` with hash-map grouping; deduplicate copies; derive subsets from existing data	High Risk	Critical
P4 Memory Efficiency	Add limits to `getTestEvents()` ; cap in-memory store growth; use cursor-based SSE fetch	High Risk	Critical
P8 Resource Utilization	Add Docker resource limits; serve production frontend build; introduce autoscaling	High Risk	Critical
P10 Build Efficiency	Add `actions/cache` for Composer, npm, and PECL extensions; move `composer install` into Dockerfile layer	High Risk	Critical
P3 API Performance	Replace SSE polling with change streams; add pagination to unbounded endpoints	Moderate	High
P9 Network Efficiency	Enable gzip in nginx; set CORS max_age; reduce redundant polling during SSE	Moderate	High
P5 CPU Efficiency	Move test simulation to queue workers; cache serialized MongoDB documents	Moderate	Medium
P6 Concurrency	Configure PHP-FPM pool sizing; add MongoDB connection pool options; use SQL aggregation	Moderate	Medium
P12 Sustainability	Replace polling with event-driven architecture; add resource limits; cache CI dependencies	Moderate	Medium

7.6 Expected Outcomes

- Replacing O(n²) `buildTree()` with hash-map grouping cuts tree-rendering latency from O(n²) to O(n), preventing latency spikes as IVR trees grow beyond demo scale.
- Adding limits to `getTestEvents()` and using cursor-based incremental fetch reduces per-SSE-poll memory allocation by 80-95%, eliminating the risk of PHP-FPM OOM kills during concurrent tests.
- Docker resource limits prevent runaway containers from starving sibling services, and production frontend serving eliminates the overhead of Vite's dev-mode file-watching and HMR.
- CI dependency caching (Composer, npm, PECL MongoDB extension) is expected to reduce build times by 60-90 seconds per run, saving ~30+ minutes of CI compute daily for an active team.
- Enabling gzip compression and CORS preflight caching reduces API response sizes by 70-80% and eliminates redundant OPTIONS requests, lowering bandwidth cost and improving perceived latency."," "stop_reason":"end_turn","session_id":"3eb72a7e-7b74-4553-bbdd-7acff735a377","total_cost_usd":3.1195115,"usage": {"input_tokens":4408,"cache_creation_input_tokens":93533,"cache_read_input_tokens":739681,"output_tokens":33097,"server_tool_use": {"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation": {"ephemeral_1h_input_tokens":93533,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations": [{"input_tokens":1,"output_tokens":1873,"cache_read_input_tokens":104188,"cache_creation_input_tokens":11169,"cache_creation": {"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":11169},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001": {"inputTokens":8741,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.008821,"contextWindow":200000,"maxOutputTokens":32000} opus-4-6": {"inputTokens":4408,"outputTokens":33097,"cacheReadInputTokens":739681,"cacheCreationInputTokens":93533,"webSearchRequests":0,"costUSD":2.1546355,"contextWindow":200000,"maxOutpu sonnet-4-6": {"inputTokens":12,"outputTokens":39133,"cacheReadInputTokens":41730,"cacheCreationInputTokens":95068,"webSearchRequests":0,"costUSD":0.9560549999999999,"contextWindow":200000,"ma [],"terminal_reason":"completed","fast_mode_state":"off","uuid":"78c82c4b-fbdb-49d4-8561-f5d7f187f47f"}

---

8. Technical Debt

EXECUTIVE SUMMARY

The FSDKC repository is a monolithic telecom-QA platform with two product modules (Discovery and Connect) running on a PHP/Laravel backend and React/TypeScript frontend, with a Node.js Express dev-API for local development. The most severe debt is in **repository hygiene**: `composer.lock` is not committed (non-reproducible PHP installs), PHPStan is declared as a dev dependency but never runs in CI or locally, and no linter, formatter, or pre-commit hook is configured anywhere in the project — code style enforcement is completely absent. The **development environment** is partially containerized via Docker Compose but the backend `.env.example` hardcodes `secret` as the database password and there is no root-level `.env.example` to unify the three sub-projects. **Database schema is managed via a single raw SQL init file** with no migration framework, no rollback path, and no per-domain ownership boundaries. On the positive side, CI does exist (GitHub Actions runs PHPUnit and frontend build), Docker Compose orchestration is functional, and the AGENTS.md files show early investment in AI-assisted development conventions. The codebase is **not yet ready** for agentic-harness adoption: the missing lock file, absent style enforcement, and manual schema management must be resolved first.

Readiness Benchmark Ratings

#	Dimension	Good	Moderate	High Risk	Measured	Rating

D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	`composer.lock` missing; PHPStan declared but never run; no linter/formatter/pre-commit hook; no CODEOWNERS or PR template (4 gaps)	High Risk
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	All 10 declared packages are imported and used in application code; PHPStan is a dev dependency but never invoked (1 unwired)	Moderate
D3	AI Tool / Agentic Readiness	ready	partial	not ready	AGENTS.md present with conventions; `.kiro/` directory scaffolded; two product modules are structurally uniform — but no CI lint gate, no CLAUDE.md, no automated scaffolding scripts	Moderate
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Single raw SQL init file for all tables; no migration framework; no rollback guards; no per-domain schema ownership; MongoDB collections have no schema validation	High Risk
D5	Development Environment	reproducible	partial	manual / fragile	Docker Compose works; `.env.example` present for backend and dev-api but not at root; no root `.env.example` unifying the three sub-projects; hardcoded `secret` password in `.env.example` and `docker-compose.yml`; no linter/formatter enforced	Moderate

No additional readiness gaps beyond the standard dimensions were observed.

8.8 Actions Required

Gap	Action	Rating	Priority
`composer.lock` not committed	Run `composer install` in the backend directory and commit the generated `composer.lock`; add `composer.lock` to CI cache key for reproducible installs	High Risk	Critical
No code style enforcement (lint/format)	Add ESLint + Prettier for frontend/dev-api, Laravel Pint or PHP-CS-Fixer for backend; create `.editorconfig` for baseline indent/whitespace rules; add lint step to `.github/workflows/ci.yml` before test/build steps	High Risk	Critical
PHPStan declared but never run	Create `backend/phpstan.neon` with level 5+ baseline; add `vendor/bin/phpstan analyse` step to CI workflow after PHPUnit; add a composer script `\"analyse\": \"phpstan analyse\"`	High Risk	Critical
No migration framework — single raw SQL init	Introduce Laravel migrations (`php artisan make:migration`); convert `docker/mariadb/init.sql` DDL into versioned migration files; add `php artisan migrate` to CI backend job and Docker `entrypoint.sh`	High Risk	Critical
No CODEOWNERS or PR template	Add `.github/CODEOWNERS` mapping `backend/` and `frontend/` to respective team leads; add `.github/pull_request_template.md` with review checklist	High Risk	High
No rollback guards on schema changes	Ensure each Laravel migration has a `down()` method; add a CI step that runs `migrate` then `migrate:rollback` to verify reversibility	High Risk	High
Hardcoded passwords in docker-compose.yml and .env.example	Replace inline `MYSQL_PASSWORD: secret` and `MYSQL_ROOT_PASSWORD: root` in `docker-compose.yml` with `env_file` directive pointing to `.env`; use placeholder values in `.env.example` (e.g. `DB_PASSWORD=changeme`)	Moderate	High
CORS wildcard on both backend and dev-api	Restrict `allowed_origins` in `backend/config/cors.php:6` from `['*']` to specific frontend origins; replace `app.use(cors())` in `dev-api/src/server.js:17` with origin-specific config	Moderate	High
Flat data ownership (no per-domain schema boundaries)	Document table ownership per module (`discovery_jobs` + `discovery_nodes` → Discovery; `connect_monitors` + `connect_check_results` → Connect); consider separate MongoDB databases per module; add FK from `discovery_nodes.parent_id` to `discovery_nodes.id`	High Risk	Medium
MongoDB collections lack schema validation	Add JSON Schema validators to `transcripts` , `test_events` , and `call_diagnostics` collections via `db.createCollection()` with `validator` option in `docker/mongodb/init.js`	High Risk	Medium
No root-level .env.example	Create a root `.env.example` that documents all required variables across the three sub-projects (backend, frontend, dev-api) with placeholder values	Moderate	Medium
No CLAUDE.md for agentic harness	Create `CLAUDE.md` at repo root documenting project conventions, build/test commands, and agent guardrails; extend existing AGENTS.md with CI-verifiable rules	Moderate	Medium
CI does not run dev-api tests	Add a `dev-api` job to `ci.yml` that installs dependencies and runs test suite (create basic API smoke tests first since none currently exist)	Moderate	Medium